CHAPITRE 2

# Premiers pas sur Python

PTSI

2026 – 2027

PLAN DU CHAPITRE


2.1 Introduction . . . 2

2.1.1 Le langage Python . . . 2

2.1.2 L'installation . . . 2

2.2 Utilisation en mode console . . . 2

2.3 Premiers programmes . . . 5

2.3.1 Où et comment s'exécute un programme Python . . . 5

2.3.2 Programmation en python . . . 5

2.3.3 Quelques exemples . . . 10

# 2.1 Introduction

## 2.1.1 Le langage Python

Python est un langage de programmation impérative, structurée et orientée objet, de haut niveau. Il présente notamment les atouts suivants :

- une syntaxe simple et concise, proche du pseudo-code algorithmique, ce qui le rend facile à apprendre : le code se lit presque comme l'algorithme lui-même ;
- un langage moderne et très répandu, aussi bien dans l'industrie que dans l'enseignement et la recherche et est porté par une large communauté de développeurs ;
- une grande richesse fonctionnelle, grâce à de nombreuses bibliothèques, dont d'excellentes bibliothèques scientifiques ;
- une bonne adéquation avec la manipulation d'objets mathématiques, sa syntaxe se rapprochant par endroits du langage mathématique ;
- la gratuité, et une disponibilité sur la quasi-totalité des plateformes (Windows, Mac, Linux, ...).

## 2.1.2 L'installation

Voir le TP1.

# 2.2 Utilisation en mode console

Nous venons de lancer Spyder et commençons par apprendre à utiliser la *console*. Python étant un langage interprété, chaque instruction saisie au prompt `>>` est exécutée dès la touche **Entrée**, et son résultat s'affiche immédiatement – inutile d'écrire un programme complet pour tester une ligne de code.

### Premiers essais

La fonction `print(.)` affiche à l'écran une chaîne de caractères (délimitée par des apostrophes `'...'` ou des guillemets `"..."`); le caractère spécial `\n` y provoque un retour à la ligne.

console

```
In [1] : print('Hello, World!')
Hello, World!

In [2] :
```

## Python, calculatrice

Utilisée sans rien d'autre, la console respecte les priorités opératoires usuelles et manipule aussi bien les entiers que les flottants. Quelques points à connaître :

- la puissance s'obtient avec `**` ; un exposant non entier est autorisé (`2**0.5` calcule ainsi $\sqrt{2}$), mais si la base est négative et l'exposant non entier, le résultat devient un **nombre complexe** (noté `x+yj`) ;
- les espaces autour des opérateurs sont facultatifs ;
- tout ce qui suit un `#` sur une ligne est un **commentaire**, ignoré par l'interpréteur – à utiliser systématiquement pour documenter son code ;
- trois opérateurs gèrent la division : `/` (division réelle), `//` (quotient de la division euclidienne) et `%` (reste), liés par la relation $n = (n \,//\, m) \times m + (n \,\%\, m)$.

console

```
In [1]: 2*4
Out[1]: 8

In [2]: 2**4
Out[2]: 16

In [3]: 47/3
Out[3]: 15.666666666666666

In [4]: 47//3
Out[4]: 15

In [5]: 47%3
Out[5]: 2

In [6]: 4**0.5
Out[6]: 2.0

In [7]:
```

## Compléter Python avec des modules

En dehors des opérateurs arithmétiques de base, Python ignore tout du vocabulaire mathématique usuel : appeler `cos(1)` ou `pi` sans plus de précision déclenche une erreur. Il faut au préalable importer le **module** correspondant, par exemple `math` pour les fonctions mathématiques courantes, ou `fractions` pour le calcul exact sur les fractions.
On peut importer une fonction précise (`from math import cos`) ou l'ensemble du module (`from math import *`).

console

```
In [1]: from math import * #
    importation des fonctions de
    la bibliothèque

In [2]: pi
Out[2]: 3.141592653589793

In [3]: cos(pi)
Out[3]: -1.0

In [4]: sqrt(2) #racine carrée
Out[4]: 1.4142135623730951

In [5]: e
Out[5]: 2.718281828459045

In [6]: exp(1)
Out[6]: 2.718281828459045

In [7]: log(e)
Out[7]: 1.0
```

## Les variables

Une variable permet de stocker une valeur en mémoire pour la réutiliser ou la modifier au fil du programme. Elle se caractérise par :

- un **identifiant** (son nom) : une chaîne alphanumérique ne débutant pas par un chiffre – on évite en pratique de la faire commencer par une majuscule, réservée aux fonctions ;
- un **contenu** muni d'un **type** (entier, flottant, chaîne, …), que la fonction `type(.)` permet d'interroger.

Une variable se crée simplement par affectation (`variable = valeur`) ; si elle existe déjà, sa valeur est écrasée.
Python autorise même de changer le type de la valeur stockée en cours de route (*typage dynamique*).
Notons enfin le raccourci `variable += expression`, équivalent à `variable = variable + expression` (et de même pour les autres opérateurs).

console

```
In [1]: a=1

In [2]: b='je suis le prof du siècle'

In [3]: a=2*a+1

In [4]: a=2*a+1

In [5]: a
Out[5]: 7

In [6]: a+=1

In [7]: print(a,b)
8 je suis le prof du siècle

In [8]: type(a)
Out[8]: int

In [9]: type(b)
Out[9]: str

In [10]:
```

## Affectations multiples et échange de variables

Python permet d'affecter plusieurs variables en une seule instruction : `a, b = 1, 2`. Les valeurs de droite sont toutes évaluées *avant* d'être affectées, ce qui rend possible l'échange des contenus de deux variables sans passer par une variable temporaire :

```
a, b = b, a
```

Cette syntaxe est notamment très pratique pour programmer des suites récurrentes, comme celle de Fibonacci ($u_0 = 0$, $u_1 = 1$, $u_{n+2} = u_{n+1} + u_n$) : `u, v = v, u+v`.

console

```
In [1]: u,v=0,1 ; print(u,v)
0 1

In [2]: u,v=v,u+v ; print(u,v)
1 1

In [3]: u,v=v,u+v ; print(u,v)
1 2

In [4]: u,v=v,u+v ; print(u,v)
2 3

In [5]: u,v=v,u+v ; print(u,v)
3 5

In [6]: u,v=v,u+v ; print(u,v)
5 8

In [7]:
```

# 2.3 Premiers programmes

## 2.3.1 Où et comment s'exécute un programme Python

Nous avons vu que Python peut s'utiliser directement dans la console : pratique pour de petits calculs ou pour tester rapidement une instruction, ce mode montre vite ses limites dès qu'un programme se complexifie, il faudrait alors retaper l'intégralité du code à chaque exécution, et tout reprendre depuis le début pour la moindre modification.

On préfère donc écrire un véritable **programme**, c'est-à-dire la suite d'instructions qui le compose, dans un **fichier texte** portant l'extension `.py`. Une fois enregistré, ce fichier peut être exécuté au sein de Spyder. Un tel environnement se compose typiquement de deux fenêtres complémentaires :

- À gauche, l'*éditeur*, où l'on rédige et enregistre le code avant de l'exécuter ;
- En bas à droite,la *console*, qui exécute le programme, affiche ses résultats, et permet d'interagir avec lui si celui-ci le demande.

temp.py

```
# -*- coding: utf-8 -*-
"""
Spyder Editor

This is a temporary script file.
"""

#JE SUIS L'EDITEUR
```

console

```
In [1]: #JE SUIS LA CONSOLE
```

## 2.3.2 Programmation en python

### Types, expressions

Les données numériques manipulées par Python relèvent de trois types : les entiers relatifs (`int`), les nombres à virgule flottante (`float`), et les nombres complexes (`complex`).

| Type | Nombres | Exemples |
|---|---|---|
| `int` | entiers relatifs | `0, 1, 2, ..., -1, -2, ...` |
| `float` | nombres à virgule | `0.1, 1.25, -2.3, 1e10 ...` |
| `complex` | nombres complexes | `1j, 1+2j, 2+1.5j, ...` |

Ces trois types se combinent grâce aux opérateurs arithmétiques usuels (`+`, `-`, `*`, `/`, `**`); les entiers disposent en outre des deux opérateurs de la division euclidienne, `//` (quotient) et `%` (reste).

Un quatrième type, la **chaîne de caractères** (`str`), regroupe toute suite de caractères encadrée par des guillemets `"..."` ou des apostrophes `'...'` :

console

```
In [1] : a = "Bonjour l'univers"
In [2] : type(a)
Out[2] : str
```

**Définition 2.1** · Expressions

On appelle *expression* toute donnée, toute variable définie, ou plus généralement le résultat d'une combinaison licite d'opérations appliquées à des variables et des données.

EXEMPLE 2.2

Si `a` désigne une variable entière, `a`, `2*a+1` ou encore `(a%2)**a` sont des expressions valides.

### Fonctions d'entrée-sorties

Un programme a souvent besoin de communiquer avec celui qui l'utilise : lui montrer un résultat, ou au contraire lui demander une information. Python met à disposition deux fonctions prédéfinies pour cela.

- `print` affiche à l'écran le contenu d'une ou plusieurs expressions qu'on lui passe en argument, séparées les unes des autres par un espace :

```
console
In [1] : a = 2
In [2] : print("Le carré de",a,"est",a**2)
Le carré de 2 est 4
```

- `input`, à l'inverse, met le programme en pause et attend que l'utilisateur tape une réponse au clavier, validée par la touche Entrée ; cette réponse est systématiquement récupérée sous forme d'une chaîne de caractères, sans qu'il soit besoin de la délimiter par des guillemets. On lui donne généralement en argument une chaîne de caractères qui sert de message d'invite, affiché avant la saisie :

```
console
In [1] : ch = input('Saisie : ')
Saisie : ma réponse
In [2] : print('Vous avez saisi :',ch)
Vous avez saisi : ma réponse
```

## Booléens et opérateurs de comparaison

**Définition 2.3** · Booléens

Le **booléen** (`bool`) est un type de données ne peut prendre que deux valeurs : `True` ou `False`.

On l'obtient naturellement en résultat d'une **comparaison** entre deux valeurs :

| | |
|---|---|
| `a < b` | `a` est strictement inférieur à `b` |
| `a <= b` | `a` est inférieur ou égal à `b` |
| `a == b` | `a` et `b` ont la même valeur |
| `a != b` | `a` et `b` ont des valeurs différentes |

(et de même, symétriquement, `a > b` et `a >= b`). Chacune de ces expressions renvoie un booléen :

console

```
In [1] : 1 < 2
Out[1]: True
In [2]: 2 <= 1
Out[2]: False
In [3]: type(1<=2)
Out[3]: bool
```

**Remarque**

**Attention** à ne pas confondre le symbole = de l'**affectation** qui crée ou modifie une variable avec ==, l'opérateur de **comparaison** qui teste une égalité et renvoie un booléen.

## Opérations sur les booléens : connecteurs logiques

Les booléens se combinent entre eux grâce à trois connecteurs logiques (par ordre de priorité décroissante) :

| | |
|---|---|
| `or` | OU logique |
| `and` | ET logique |
| `not` | NON logique |

console

```
In [1]: a = (1 <= 2) ; b = (a == False) ; print(a,b)
True False
In [2]: print(a or b, a and b, not(a), not(b))
True False False True
In [3]: var = "chaine"; var == "Chaine"
Out[3]: False
In [4]: print(var != "Chaine", not(var == "Chaine"))
True True
```

Notons enfin que la comparaison == porte sur la **valeur**, indépendamment du type exact (`int` ou `float`) ou du délimiteur utilisé pour une chaîne (guillemets ou apostrophes) :

console

```
In [5]: (1 == 1.0) and ("chapeau" == 'chapeau')
Out[5]: True
```

## Boucle `for`

La boucle `for` répète un bloc d'instructions un nombre de fois connu à l'avance, en faisant parcourir à une variable une séquence de valeurs produite par `range` :

- ▷ `for variable in range(N):` – `variable` parcourt $0, 1, \ldots, N-1$ ;
- ▷ `for variable in range(n,m):` – `variable` parcourt $n, n+1, \ldots, m-1$ ;
- ▷ `for variable in range(n,m,k):` – `variable` parcourt $n, n+k, n+2k, \ldots$ jusqu'à $m-1$ (pas de $k$).

Dans les trois cas, la syntaxe est identique : le mot-clé `for`, le nom de la variable, `in range(...)`, deux-points, puis un bloc d'instructions uniformément indenté (même principe que pour une fonction).

EXEMPLE 2.4

Calcul et affichage de la somme des entiers de 0 à 1000 :

somme.py

```
S = 0
for k in range(1001):
    S += k
print(S)
```

EXEMPLE 2.5

Somme des 1000 premiers termes de la suite de Fibonacci :

fibonacci.py

```
N = 1000
u, v = 0, 1
S = u+v
for k in range(N-2):
    u, v = v, u+v
    S += v
print(S)
```

### Boucle `while`

L'instruction `while` *condition* répète un bloc d'instructions tant que *condition* reste vérifiée – contrairement à `for`, le nombre de répétitions n'est pas connu à l'avance : c'est la condition, réévaluée à chaque tour, qui décide de la poursuite ou de l'arrêt de la boucle.
*condition* est le plus souvent une expression booléenne.

```
while condition:
    Instruction1
    ...
    Derniere instruction
```

EXEMPLE 2.6

Soit $(u_n)$ une suite géométrique de raison $q > 1$ et de premier terme positif donc de limite $+\infty$. On cherche le premier rang $n$ à partir duquel $u_n$ dépasse un seuil donné :

seuil.py

```
u = 1          # premier terme u_0
q = 1.05       # raison de la suite
seuil = 1000
n = 0
while u <= seuil:
    u *= q
    n += 1
print("Rang a partir duquel la
    suite depasse", seuil, ":", n)
```

console

```
Rang à partir duquel la suite dé
   passe 1000 : 142
```

### Structure de test : `if ... [else]`

La plus simple des structures de test est l'instruction **`if ... [else]`**, qui permet d'exécuter un bloc d'instructions ou un autre selon qu'une condition est vérifiée ou non.

La *condition* est évaluée comme un booléen. Si elle vaut **`True`**, seul le premier bloc s'exécute ; si elle vaut **`False`**, seul le second s'exécute (lorsqu'il est présent).

Dans les deux cas, l'exécution du programme se poursuit ensuite normalement.

Le bloc **`else`** est optionnel : en son absence, rien ne se passe simplement lorsque la condition est fausse.

```
if condition:
    Bloc d'instructions (si condition vraie)
else:
    Bloc d'instructions (si condition fausse)
```

EXEMPLE 2.7

signe.py

```
a = float(input('Saisissez un nombre'))
if a >= 0:
    print(a, 'est positif')
else:
    print(a, 'est negatif')
```

console

```
Saisissez un nombre-3.5
-3.5 est negatif
```

Lorsqu'il faut distinguer plus de deux cas, il devient vite lourd d'imbriquer des **`if`**/**`else`** les uns dans les autres. Python propose pour cela le mot-clé **`elif`** (contraction de *else if*), qui permet d'enchaîner plusieurs conditions à la suite.

Les conditions sont testées **dans l'ordre** : dès que l'une d'elles est vérifiée, le bloc correspondant s'exécute et **tous les autres sont ignorés** (y compris les conditions suivantes, qui ne sont même pas évaluées).

Le bloc **`else`** final, toujours optionnel, ne s'exécute que si aucune des conditions précédentes n'a été vérifiée.

```
if condition1:
    Bloc d'instructions 1
elif condition2:
    Bloc d'instructions 2
elif condition3:
    Bloc d'instructions 3
else:
    Bloc d'instructions 4
```

EXEMPLE 2.8

signe.py

```
a = float(input('Saisissez un nombre'))
if a > 0:
    print(a, 'est positif')
elif a < 0:
    print(a, 'est negatif')
else:
    print(a, 'est nul')
```

console

```
Saisissez un nombre0
0.0 est nul
```

### Fonctions

**Définition 2.9** · Fonction

Une *fonction* est un bloc d'instructions nommé, défini une fois pour toutes à l'aide du mot-clé `def`, que l'on peut ensuite appeler autant de fois que nécessaire sans avoir à le réécrire.

- ▷ les valeurs qu'on lui transmet en entrée, entre parenthèses lors de l'appel, sont ses **paramètres** (ou *arguments*);
- ▷ la valeur qu'elle renvoie en sortie, précisée par l'instruction `return`, est son **résultat**. Cette instruction met immédiatement fin à l'exécution de la fonction : tout ce qui suit un `return` atteint n'est pas exécuté.

Une fonction peut n'avoir aucun paramètre, ou plusieurs; elle peut aussi ne renvoyer aucun résultat, auquel cas on parle plutôt de *procédure*.

Lorsqu'un programme contient des définitions de fonctions, son exécution a pour seul effet de **définir** ces fonctions (le corps de la fonction n'est pas exécuté à ce stade) :

tmp 1

```
## definition de la fonction valeur
    absolue vabs

def vabs(x):
    if x >= 0:
        return x
    else:
        return -x
```

console

```
In [1]: (executing lines 1 to
    7 of "<tmp 1>")

In [2]:
```

Une fois définie, la fonction peut alors être appelée depuis la console :

console

```
In [1]: (executing lines 1 to 7 of "<tmp 1>")

In [2]: vabs(-1.5)
Out[2]: 1.5

In [3]: vabs(3)
Out[3]: 3

In [4]:
```

## 2.3.3 Quelques exemples

Pour clore ce premier chapitre, voici quelques exemples simples qui mobilisent l'essentiel de ce qui vient d'être vu : boucles `for` et `while`, tests `if`/`elif`/`else`, booléens et fonctions.

EXEMPLE 2.10

Écrire une fonction qui teste si un entier est pair ou impair, à l'aide de l'opérateur `%` :

parite.py

```
1 def parite(n):
2     if n % 2 == 0:
3         return 'pair'
4     else:
5         return 'impair'
```

console

```
In [1]: parite(17)
Out[1]: 'impair'

In [2]: parite(12)
Out[2]: 'pair'
```

EXEMPLE 2.11

Écrire une fonction qui calcule la somme des chiffres d'un entier, à l'aide d'une boucle `while` (on extrait le chiffre des unités avec `% 10`, puis on l'enlève avec une division entière par `10`) :

sommechiffres.py

```
1 def somme_chiffres(n):
2     s = 0
3     while n > 0:
4         s += n % 10
5         n //= 10
6     return s
```

console

```
In [1]: somme_chiffres(12345
    )
Out[1]: 15
```

Attention : cette fonction suppose que `n` est bien un entier positif. Si on lui passe autre chose, Python lève une erreur dès la comparaison `n > 0`, puisque comparer un `str` à un `int` n'a pas de sens :

console

```
In [2]: somme_chiffres('12345')
TypeError: '>' not supported between instances of 'str' and 'int'
```

EXEMPLE 2.12

Afficher la table de multiplication d'un entier, à l'aide d'une boucle `for` :

table.py

```
1 n = 7
2 for k in range(1, 11):
3     print(n, 'x', k, '=', n*k)
```

console

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

EXEMPLE 2.13

Écrire une fonction qui teste si un entier est premier, en combinant `for`, `if` et un `return` booléen :

premier.py

```python
def est_premier(n):
    if n < 2:
        return False
    for k in range(2, n):
        if n % k == 0:
            return False
    return True
```

console

```
In [1]: est_premier(20)
Out[1]: False

In [2]: est_premier(29)
Out[2]: True
```